

Functions — Code একবার লিখে বারবার ব্যবহার করার উপায়

JavaScript শেখার ক্ষেত্রে **Function** একটি অত্যন্ত গুরুত্বপূর্ণ concept। তুমি যদি JavaScript দিয়ে:

- Website
- Node.js Backend
- Calculator
- Web App
- API
- E-commerce
- React Application
- Authentication
- Dashboard

ইত্যাদি বানাতে চাও, তাহলে Function ভালোভাবে বুঝতেই হবে।

PART 01 — CORE FUNCTION MECHANICS, PARAMETERS & ARGUMENTS (6.1 – 6.12)

6.1 What is a Function?

Function হলো এমন একটি reusable block of code, যেটা একটি নির্দিষ্ট কাজ করার জন্য তৈরি করা হয়।

সহজভাবে:

Core Concept: একবার code লিখবে → প্রয়োজন অনুযায়ী অনেকবার call করবে।

REAL-LIFE EXAMPLE

ধরো তোমার বাসায় একটি **washing machine** আছে।

তুমি শুধু:

Start

চাপ দিলে machine-এর ভিতরে অনেক কাজ automatically হয়।

Programming-এ function অনেকটা এমন:

```
Function
  ↓
Input
  ↓
Process
  ↓
Output
```

6.2 Without Function

ধরো আমাদের 3 জনের জন্য greeting দেখাতে হবে।

JAVASCRIPT

CODE

```
console.log("Hello Shariar");
console.log("Hello Rahim");
console.log("Hello Karim");
```

এটা কাজ করবে।

কিন্তু একই ধরনের কাজ অনেক জায়গায় করতে হলে code repeat করতে হবে।

6.3 With Function

JAVASCRIPT

CODE

```
function greet(name) {  
  console.log("Hello " + name);  
}  
  
greet("Shariar");  
greet("Rahim");  
greet("Karim");
```

OUTPUT

Hello Shariar Hello Rahim Hello Karim

একবার function লিখেছি:

JAVASCRIPT

CODE

```
function greet(name) {  
  console.log("Hello " + name);  
}
```

তারপর প্রয়োজন অনুযায়ী call করেছি:

JAVASCRIPT

CODE

```
greet("Shariar");  
greet("Rahim");  
greet("Karim");
```

এটাই **Code Reusability**!

6.4 Function-এর Basic Structure

JAVASCRIPT

CODE

```
function functionName(parameter) {  
  // code  
}
```

উদাহরণ:

JAVASCRIPT

CODE

```
function greet(name) {  
  console.log("Hello " + name);  
}
```

এখানে:

অংশ	অর্থ
<code>function</code>	Function declare করার keyword
<code>greet</code>	Function-এর নাম
<code>name</code>	Parameter
<code>{ }</code>	Function body
<code>console.log()</code>	Function-এর কাজ

6.5 Function Declaration

এটাকে বলা হয় **Function Declaration**।

JAVASCRIPT

CODE

```
function sayHello() {  
  console.log("Hello World!");  
}
```

এখন function তৈরি হয়েছে।

কিন্তু এখনো execute হয়নি।

Execute করতে হবে:

JAVASCRIPT

CODE

```
sayHello();
```

OUTPUT

```
Hello World!
```

6.6 Function Call / Invocation

Function execute করাকে বলা হয়:

- Function Call
- Function Invocation

উদাহরণ:

JAVASCRIPT

CODE

```
function greet() {  
  console.log("Hello!");  
}  
  
greet();
```

এখানে:

JAVASCRIPT

CODE

```
greet();
```

হলো function call।

6.7 Function একাধিকবার Call করা

একটি function একবার লিখে অনেকবার ব্যবহার করা যায়।

JAVASCRIPT

CODE

```
function greet() {  
  console.log("Hello!");  
}  
  
greet();  
greet();  
greet();
```

OUTPUT

```
Hello! Hello! Hello!
```

6.8 Parameter

Function-এর ভিতরে input নেওয়ার জন্য **parameter** ব্যবহার করি।

JAVASCRIPT

CODE

```
function greet(name) {  
  console.log("Hello " + name);  
}
```

এখানে:

JAVASCRIPT

CODE

```
name
```

হলো parameter।

6.9 Argument

Function call করার সময় যে actual value পাঠানো হয় তাকে **argument** বলে।

JAVASCRIPT

CODE

```
greet("Shariar");
```

এখানে:

```
name → parameter  
"Shariar" → argument
```

সহজভাবে

```
Parameter → Function তৈরি করার সময়  
Argument → Function call করার সময়
```

6.10 Multiple Parameters

একটি function-এ একাধিক parameter থাকতে পারে।

JAVASCRIPT

CODE

```
function add(a, b) {  
  console.log(a + b);  
}  
  
add(10, 20);
```

OUTPUT

```
30
```

এখানে:

```
a = 10  
b = 20
```

6.11 তিনটি Parameter

JAVASCRIPT

CODE

```
function introduce(name, age, country) {  
  console.log(  
    "Name:", name,  
    "Age:", age,  
    "Country:", country  
  );  
}  
  
introduce("Shariar", 23, "Bangladesh");
```

OUTPUT

```
Name: Shariar Age: 23 Country: Bangladesh
```

6.12 Function-এর ভিতরে Calculation

Function শুধু message print করার জন্য নয়।

JAVASCRIPT

CODE

```
function add(a, b) {  
  console.log(a + b);  
}  
  
add(15, 25);
```

OUTPUT

```
40
```

আর:

JAVASCRIPT

CODE

```
function multiply(a, b) {  
  console.log(a * b);  
}  
  
multiply(5, 10);
```

OUTPUT

```
50
```

6.13 **return**

Function-এর সবচেয়ে গুরুত্বপূর্ণ বিষয়গুলোর একটি হলো **return**।

return ব্যবহার করে function থেকে কোনো value বাইরে পাঠানো যায়।

JAVASCRIPT

CODE

```
function add(a, b) {  
  return a + b;  
}  
  
let result = add(10, 20);  
  
console.log(result);
```

OUTPUT

30

এখানে:

JAVASCRIPT

CODE

```
return a + b;
```

মানে function result বাইরে পাঠিয়ে দিচ্ছে।

6.14 console.log() বনাম return

এটা খুব গুরুত্বপূর্ণ।

`console.log()`

সুধু output console-এ দেখায়।

```
JAVASCRIPT CONSOLE.LOG()  
  
function add(a, b) {  
    console.log(a + b);  
}  
  
let result = add(10, 20);  
  
console.log(result);
```

OUTPUT

30 undefined

কারণ function কিছু `return` করেনি।

`return`

Value ফেরত দেয়।

```
JAVASCRIPT RETURN  
  
function add(a, b) {  
    return a + b;  
}  
  
let result = add(10, 20);  
  
console.log(result);
```

Output:

30

মনে রাখার নিয়ম

```
console.log()  
→ দেখায়  
  
return  
→ value ফেরত দেয়
```

6.15 return Function Execution বন্ধ করে

```
JAVASCRIPT CODE  
  
function test() {  
    console.log("Before");  
  
    return;  
  
    console.log("After");  
}  
  
test();
```

OUTPUT

Before

`return` হওয়ার পর function আর নিচের code execute করে না।

6.16 Return Value Store করা

JAVASCRIPT

CODE

```
function square(number) {  
    return number * number;  
}  
  
let result = square(5);  
  
console.log(result);
```

OUTPUT

25

6.17 Return Value নিয়ে আরও কাজ করা

JAVASCRIPT

CODE

```
function add(a, b) {  
    return a + b;  
}  
  
let result = add(10, 20);  
  
console.log(result * 2);
```

OUTPUT

60

কারণ:

```
add(10, 20)  
→ 30  
  
30 * 2  
→ 60
```

6.18 Function-এর ভিতরে Function Call

JAVASCRIPT

CODE

```
function add(a, b) {  
    return a + b;  
}  
  
function double(number) {  
    return number * 2;  
}  
  
let result = double(add(10, 20));  
  
console.log(result);
```

OUTPUT

60

Step:

```
add(10, 20)  
↓  
30  
  
double(30)  
↓  
60
```

6.19 Function-এর Default Parameter

কখনো argument না দিলে default value ব্যবহার করতে চাই।

JAVASCRIPT

CODE

```
function greet(name = "Guest") {  
  console.log("Hello " + name);  
}  
  
greet();
```

OUTPUT

Hello Guest

কিন্তু:

JAVASCRIPT

CODE

```
greet("Shariar");
```

Output:

Hello Shariar

6.20 Multiple Default Parameters

JAVASCRIPT

CODE

```
function introduce(  
  name = "Guest",  
  country = "Unknown"  
) {  
  console.log(name, country);  
}  
  
introduce();
```

OUTPUT

Guest Unknown

আর:

JAVASCRIPT

CODE

```
introduce("Shariar", "Bangladesh");
```

Output:

Shariar Bangladesh

6.21 Function Expression

Function declaration ছাড়াও function variable-এর মধ্যে রাখা যায়।

JAVASCRIPT

CODE

```
const greet = function () {
  console.log("Hello!");
};

greet();
```

OUTPUT

Hello!

এটাকে বলা হয়:

Core Concept: Function Expression

6.22 Function Declaration বনাম Function Expression**Function Declaration**

JAVASCRIPT

FUNCTION DECLARATION

```
function greet() {
  console.log("Hello");
}
```

Function Expression

JAVASCRIPT

FUNCTION EXPRESSION

```
const greet = function () {
  console.log("Hello");
};
```

দুটোই function।

কিন্তু syntax আলাদা।

6.23 Function Declaration Hoisting

Function declaration-এর একটি বিশেষ feature আছে।

তুমি function call আগে করলেও কাজ করতে পারে।

JAVASCRIPT

CODE

```
greet();

function greet() {
  console.log("Hello!");
}
```

OUTPUT

Hello!

কারণ function declaration hoisted হয়।

⚠ Hoisting আমরা Scope & Hoisting chapter-এ বিস্তারিতভাবে দেখব।

6.24 Function Expression আগে Call করলে

JAVASCRIPT

CODE

```
greet();

const greet = function () {
  console.log("Hello!");
};
```

এটি কাজ করবে না এবং error হবে।

কারণ `const` variable initialization-এর আগে access করা যায় না।

6.25 Arrow Function

Modern JavaScript-এ সবচেয়ে বেশি ব্যবহৃত syntaxগুলোর একটি হলো **Arrow Function**।

Traditional:

JAVASCRIPT

TRADITIONAL:

```
function add(a, b) {
  return a + b;
}
```

Arrow:

JAVASCRIPT

ARROW:

```
const add = (a, b) => {
  return a + b;
};
```

দুটোই:

```
10 + 20 = 30
```

return করবে।

6.26 Arrow Function Short Syntax

যদি function body-তে শুধু একটি expression return করতে হয়:

JAVASCRIPT

CODE

```
const add = (a, b) => a + b;
```

এখানে:

JAVASCRIPT

CODE

```
return
```

লিখতে হয় না।

EXAMPLE

JAVASCRIPT

EXAMPLE

```
const square = number => number * number;

console.log(square(5));
```

OUTPUT

```
25
```

6.27 Arrow Function with One Parameter

একটি parameter থাকলে parentheses বাদ দেওয়া যায়।

JAVASCRIPT

CODE

```
const double = number ⇒ number * 2;  
  
console.log(double(5));
```

OUTPUT

10

তবে parentheses দিয়েও লেখা যায়:

JAVASCRIPT

CODE

```
const double = (number) ⇒ number * 2;
```

দুটোই valid।

6.28 No Parameter Arrow Function

JAVASCRIPT

CODE

```
const greet = () ⇒ {  
  console.log("Hello!");  
};  
  
greet();
```

OUTPUT

Hello!

6.29 Multiple Parameters

JAVASCRIPT

CODE

```
const add = (a, b) ⇒ a + b;  
  
console.log(add(10, 20));
```

OUTPUT

30

6.30 Arrow Function with Multiple Lines

JAVASCRIPT

CODE

```
const calculate = (a, b) ⇒ {  
  let sum = a + b;  
  let result = sum * 2;  
  
  return result;  
};  
  
console.log(calculate(10, 20));
```

OUTPUT

60

6.31 Function-এর বিভিন্ন ধরন

এখন পর্যন্ত আমরা দেখেছি:

Function Declaration

JAVASCRIPT

FUNCTION DECLARATION

```
function greet() {
  console.log("Hello");
}
```

Function Expression

JAVASCRIPT

FUNCTION EXPRESSION

```
const greet = function () {
  console.log("Hello");
};
```

Arrow Function

JAVASCRIPT

ARROW FUNCTION

```
const greet = () => {
  console.log("Hello");
};
```

6.32 Anonymous Function

যে function-এর কোনো নাম নেই তাকে **Anonymous Function** বলে।

JAVASCRIPT

CODE

```
const greet = function () {
  console.log("Hello");
};
```

এখানে function-এর নিজস্ব কোনো নাম নেই।

Function-টি `greet` variable-এর মধ্যে রাখা হয়েছে।

6.33 IIFE — Immediately Invoked Function Expression

IIFE এমন function যেটা তৈরি হওয়ার সঙ্গে সঙ্গে execute হয়।

JAVASCRIPT

CODE

```
(function () {
  console.log("Hello!");
})();
```

OUTPUT

Hello!

এখানে function আলাদা করে call করা হয়নি।

তৈরি করার সঙ্গে সঙ্গেই execute হয়েছে।

6.34 IIFE with Parameter

JAVASCRIPT

CODE

```
(function (name) {
  console.log("Hello " + name);
})("Shariar");
```

OUTPUT

Hello Shariar

6.35 Why IIFE?

পুরনো JavaScript code-এ IIFE ব্যবহার করা হতো private scope তৈরি করার জন্য।

JAVASCRIPT

CODE

```
(function () {  
  let secret = "12345";  
  
  console.log(secret);  
})();
```

`secret` বাইরে accessible হবে না।

Modern JavaScript-এ modules এবং block scope-এর কারণে IIFE আগের মতো বেশি প্রয়োজন হয় না, কিন্তু legacy code বুঝতে এটি জানা দরকার।

6.36 Function as a Value

JavaScript-এ function শুধু function নয়, একটি **value** হিসেবেও ব্যবহার করা যায়।

JAVASCRIPT

CODE

```
const greet = function () {  
  console.log("Hello");  
};
```

এখানে function-টি variable-এর value।

এজন্য function-কে বলা হয়:

Core Concept: First-Class Function

6.37 First-Class Functions

JavaScript-এ function:

- Variable-এ রাখা যায়
- অন্য function-এ পাঠানো যায়
- Function থেকে return করা যায়
- Array/Object-এর মধ্যে রাখা যায়

EXAMPLE

JAVASCRIPT

EXAMPLE

```
const greet = function () {  
  console.log("Hello");  
};  
  
const anotherFunction = greet;  
  
anotherFunction();
```

OUTPUT

Hello

6.38 Function as Argument

একটি function-এর argument হিসেবে আরেকটি function পাঠানো যায়।

JAVASCRIPT

CODE

```
function greet(name) {
  console.log("Hello " + name);
}

function processUser(callback) {
  callback("Shariar");
}

processUser(greet);
```

OUTPUT

Hello Shariar

এখানে:

JAVASCRIPT

CODE

greet

কে পাঠানো হয়েছে:

JAVASCRIPT

CODE

processUser(greet);

6.39 Callback Function

যে function-কে অন্য function-এর argument হিসেবে পাঠানো হয় তাকে **Callback Function** বলে।

JAVASCRIPT

CODE

```
function greet(name) {
  console.log("Hello " + name);
}

function processUser(callback) {
  callback("Shariar");
}

processUser(greet);
```

এখানে:

```
greet
↓
callback
↓
processUser()
```

greet হলো callback function।

6.40 Callback-এর Real-Life Example

ধরো restaurant-এ তুমি খাবার order দিলে।

```
Order placed
  ↓
Wait
  ↓
Food ready
  ↓
Notify customer
```

Programming-এ:

JAVASCRIPT

CODE

```
function orderFood(callback) {
  console.log("Food ordered.");

  callback();
}

function foodReady() {
  console.log("Food is ready!");
}

orderFood(foodReady);
```

OUTPUT

```
Food ordered. Food is ready!
```

এটা asynchronous JavaScript বোঝার foundation।

6.41 Callback with Arrow Function

JAVASCRIPT

CODE

```
function processUser(callback) {
  callback("Shariar");
}

processUser((name) => {
  console.log("Hello " + name);
});
```

OUTPUT

```
Hello Shariar
```

6.42 Higher-Order Function

যে function:

1. অন্য function গ্রহণ করে অথবা 2. অন্য function return করে

তাকে **Higher-Order Function** বলা হয়।

EXAMPLE

JAVASCRIPT

EXAMPLE

```
function process(callback) {
  callback();
}

process(() => {
  console.log("Hello");
});
```

`process()` একটি higher-order function।

কারণ এটি function argument হিসেবে গ্রহণ করছে।

6.43 Function Returning Function

JAVASCRIPT

CODE

```
function outer() {  
  
  function inner() {  
    console.log("Hello from inner");  
  }  
  
  return inner;  
}  
  
const result = outer();  
  
result();
```

OUTPUT

Hello from inner

এখানে `outer()` একটি function return করছে।

এটিও higher-order function-এর example।

6.44 Closure-এর Basic Idea

এখানে আমরা closure-এর পুরো chapter করছি না, কিন্তু function-এর সঙ্গে এর connection বুঝে রাখা।

JAVASCRIPT

CODE

```
function outer() {  
  
  let message = "Hello";  
  
  function inner() {  
    console.log(message);  
  }  
  
  return inner;  
}  
  
const greet = outer();  
  
greet();
```

OUTPUT

Hello

`outer()` execute শেষ হওয়ার পরও `inner()` তার outer variable `message` access করতে পারছে।

এটাই **Closure**-এর basic idea।

Closure পরে আলাদা advanced chapter-এ বিস্তারিত হবে।

6.45 Rest Parameter

Function-এ কতগুলো argument আসবে জানা না থাকলে **rest parameter** ব্যবহার করা যায়।

Syntax:

JAVASCRIPT

CODE

```
function functionName( ... values) {  
}
```

EXAMPLE

JAVASCRIPT

EXAMPLE

```
function addAll( ... numbers) {  
  console.log(numbers);  
}  
  
addAll(10, 20, 30, 40);
```

OUTPUT

```
[10, 20, 30, 40]
```

`... numbers` সব arguments-কে একটি array-এর মধ্যে নিয়ে এসেছে।

6.46 Rest Parameter দিয়ে Sum

JAVASCRIPT

CODE

```
function addAll( ... numbers) {  
  
  let sum = 0;  
  
  for (let number of numbers) {  
    sum += number;  
  }  
  
  return sum;  
}  
  
console.log(addAll(10, 20, 30, 40));
```

OUTPUT

```
100
```

6.47 Rest Parameter vs Spread Operator

দুটো দেখতে একই:

JAVASCRIPT

CODE

...

কিন্তু কাজের জায়গা আলাদা।

Rest

Arguments **collect** করে:

JAVASCRIPT

REST

```
function add( ... numbers) {  
  console.log(numbers);  
}
```

Spread

Values **spread** করে:

JAVASCRIPT

SPREAD

```
let numbers = [10, 20, 30];  
  
console.log( ... numbers);
```

Output:

10 20 30

এটা আমরা Destructuring, Spread & Rest chapter-এ আরও বিস্তারিত করব।

6.48 Function-এর Argument কম দিলে কী হয়?

JAVASCRIPT

CODE

```
function add(a, b) {  
  return a + b;  
}  
  
console.log(add(10));
```

OUTPUT

NaN

কারণ:

```
a = 10  
b = undefined
```

তাই:

JAVASCRIPT

CODE

10 + undefined

→ NaN

6.49 Default Parameter দিয়ে Solve

JAVASCRIPT

CODE

```
function add(a, b = 0) {  
  return a + b;  
}  
  
console.log(add(10));
```

OUTPUT

10

6.50 Extra Argument দিলে কী হয়?

JAVASCRIPT

CODE

```
function add(a, b) {  
  return a + b;  
}  
  
console.log(add(10, 20, 30));
```

OUTPUT

30

এখানে 10 → a

20 → b

30 কোনো parameter পায়নি।

JavaScript সাধারণভাবে extra arguments ignore করে, যদি না `arguments` বা rest parameter দিয়ে access করা হয়।

6.51 `arguments` Object

Traditional function-এর ভিতরে `arguments` নামে একটি special object-like value থাকে।

JAVASCRIPT

CODE

```
function test(a, b) {  
  console.log(arguments);  
}  
  
test(10, 20, 30);
```

এতে passed arguments দেখতে পারবে।

⚠ Arrow function-এর নিজের `arguments` নেই।

Modern JavaScript-এ variable number of arguments-এর জন্য সাধারণত rest parameter:

JAVASCRIPT

CODE

```
function test(...args) {  
  console.log(args);  
}
```

বেশি পরিষ্কার।

6.52 Pure Function

একটি function যদি:

- একই input দিলে সবসময় একই output দেয়
- বাইরের state পরিবর্তন না করে

তাহলে তাকে **Pure Function** বলা হয়।

EXAMPLE

JAVASCRIPT

EXAMPLE

```
function add(a, b) {
  return a + b;
}
```

```
add(10, 20) → 30
add(10, 20) → 30
```

সবসময় একই result।

6.53 Impure Function

যদি function বাইরের data পরিবর্তন করে বা external state-এর ওপর depend করে, তাহলে pure নয়।

JAVASCRIPT

CODE

```
let total = 0;

function addToTotal(amount) {
  total += amount;
}
```

Function বাইরের `total` পরিবর্তন করছে।

তাই এটি pure function নয়।

6.54 Function Scope

Function-এর ভিতরে declare করা variable সাধারণত function-এর বাইরে accessible নয়।

JAVASCRIPT

CODE

```
function test() {
  let message = "Hello";

  console.log(message);
}

test();
```

কাজ করবে।

কিন্তু:

JAVASCRIPT

CODE

```
console.log(message);
```

করলে error হবে।

কারণ `message` function scope-এর ভিতরে।

6.55 Local Variable

Function-এর ভিতরে তৈরি variable-কে সাধারণভাবে local variable বলা হয়।

JAVASCRIPT

CODE

```
function calculate() {  
  let result = 100;  
  
  console.log(result);  
}
```

`result` বাইরে ব্যবহার করা যাবে না।

6.56 Global Variable

Function-এর বাইরে declare করা variable function-এর ভিতর থেকে access করা যায়।

JAVASCRIPT

CODE

```
let name = "Shariar";  
  
function greet() {  
  console.log(name);  
}  
  
greet();
```

OUTPUT

Shariar

Function বাইরের `name` access করতে পারছে।

6.57 Function Parameter Scope

JAVASCRIPT

CODE

```
function greet(name) {  
  console.log(name);  
}  
  
greet("Shariar");
```

`name` function-এর local scope-এর অংশ।

Function-এর বাইরে:

JAVASCRIPT

CODE

```
console.log(name);
```

কাজ করবে না।

6.58 Function Composition

একটি function-এর output অন্য function-এর input হিসেবে ব্যবহার করা যায়।

JAVASCRIPT

CODE

```
function double(number) {  
  return number * 2;  
}  
  
function addFive(number) {  
  return number + 5;  
}  
  
let result = addFive(double(10));  
  
console.log(result);
```

OUTPUT

25

Step:

```
double(10)  
↓  
20  
  
addFive(20)  
↓  
25
```

6.59 Recursive Function

যখন একটি function নিজেকেই call করে তাকে **Recursion** বলে।

EXAMPLE

JAVASCRIPT

EXAMPLE

```
function countdown(number) {  
  if (number === 0) {  
    return;  
  }  
  
  console.log(number);  
  
  countdown(number - 1);  
}  
  
countdown(5);
```

OUTPUT

5 4 3 2 1

এখানে:

JAVASCRIPT

CODE

```
countdown()
```

নিজেেকেই call করছে।

6.60 Recursion-এর Base Case

Recursion-এ সবচেয়ে গুরুত্বপূর্ণ বিষয় হলো **Base Case**।

JAVASCRIPT

CODE

```
if (number === 0) {  
  return;  
}
```

এটা না থাকলে function নিজেকে call করতেই থাকবে এবং eventually:

```
Maximum call stack size exceeded
```

এর মতো error হতে পারে।

6.61 Factorial using Recursion

Mathematics:

```
5! = 5 × 4 × 3 × 2 × 1  
    = 120
```

JavaScript:

JAVASCRIPT

CODE

```
function factorial(n) {  
  
  if (n === 1) {  
    return 1;  
  }  
  
  return n * factorial(n - 1);  
}  
  
console.log(factorial(5));
```

OUTPUT

```
120
```

Step:

```
5 × factorial(4)  
↓  
5 × 4 × factorial(3)  
↓  
5 × 4 × 3 × factorial(2)  
↓  
5 × 4 × 3 × 2 × factorial(1)  
↓  
120
```

6.62 Function-এর Best Practices

1. Function-এর নাম meaningful রাখো

ভালো:

```
JAVASCRIPT
```

1. FUNCTION-এর নাম MEANINGFUL রাখো

```
calculateTotal()
```

খারাপ:

```
JAVASCRIPT
```

1. FUNCTION-এর নাম MEANINGFUL রাখো

```
x()
```

2. একটি function ideally একটি clear কাজ করুক

ভালো:

```
JAVASCRIPT
```

2. একটি FUNCTION IDEALLY একটি CLEAR কাজ করুক

```
calculateTotal()
```

আরেকটি:

```
JAVASCRIPT
```

2. একটি FUNCTION IDEALLY একটি CLEAR কাজ করুক

```
validateUser()
```

এক function-এ সবকিছু ঢুকিয়ে না দেওয়া ভালো।

3. Repeated code function-এ রাখো

যদি একই code বারবার লিখতে হয়:

```
Stop!  
↓  
Function বানাও
```

4. খুব বড় function এড়িয়ে চলো

যেমন:

```
JAVASCRIPT
```

4. খুব বড় FUNCTION এড়িয়ে চলো

```
function doEverything() {  
  // login  
  // database  
  // payment  
  // email  
  // UI  
  // validation  
  // ...  
}
```

এর বদলে ছোট ছোট function:

```
JAVASCRIPT
```

4. খুব বড় FUNCTION এড়িয়ে চলো

```
validateUser();  
processPayment();  
sendEmail();  
updateUI();
```

6.63 Real-Life Mini Project — Calculator

JAVASCRIPT

CODE

```
function add(a, b) {  
    return a + b;  
}  
  
function subtract(a, b) {  
    return a - b;  
}  
  
function multiply(a, b) {  
    return a * b;  
}  
  
function divide(a, b) {  
    return a / b;  
}  
  
console.log(add(10, 5));  
console.log(subtract(10, 5));  
console.log(multiply(10, 5));  
console.log(divide(10, 5));
```

OUTPUT

```
15 5 50 2
```

এখানে calculator-এর প্রতিটি operation আলাদা function।

এটাই clean code-এর basic ideal

6.64 Calculator with Operator

এবার function-এর ভিতরে `switch` ব্যবহার করি।

JAVASCRIPT

CODE

```
function calculate(a, b, operator) {  
  
    switch (operator) {  
  
        case "+":  
            return a + b;  
  
        case "-":  
            return a - b;  
  
        case "*":  
            return a * b;  
  
        case "/":  
            return a / b;  
  
        default:  
            return "Invalid operator";  
    }  
}  
  
console.log(calculate(10, 5, "+"));  
console.log(calculate(10, 5, "*"));
```

OUTPUT

```
15 50
```

🔥 এখানে আমরা Chapter 4 + Chapter 5 + Chapter 6 একসাথে ব্যবহার করেছি।

6.65 Real-Life Mini Project — Shopping Cart

JAVASCRIPT

CODE

```
function calculateTotal(cart) {  
  
    let total = 0;  
  
    for (let product of cart) {  
        total += product.price;  
    }  
  
    return total;  
}  
  
let cart = [  
    {  
        name: "Keyboard",  
        price: 1500  
    },  
    {  
        name: "Mouse",  
        price: 800  
    },  
    {  
        name: "Headphone",  
        price: 2000  
    }  
];  
  
let total = calculateTotal(cart);  
  
console.log("Total:", total);
```

OUTPUT

Total: 4300

এখানে:

```
Function  
↓  
Array  
↓  
for ... of  
↓  
Object  
↓  
return
```

সব একসাথে কাজ করছে।

6.66 Function-এর Complete Flow

এখন একটা function-এর পুরো lifecycle দেখা:

```
Function Definition
  ↓
Parameters
  ↓
Function Call
  ↓
Arguments
  ↓
Function Body
  ↓
Processing
  ↓
return
  ↓
Result
```

Example:

JAVASCRIPT

CODE

```
function add(a, b) {
  return a + b;
}

let result = add(10, 20);
```

Flow:

```
function add()
  ↓
a, b
  ↓
add(10, 20)
  ↓
a = 10, b = 20
  ↓
10 + 20
  ↓
return 30
  ↓
result = 30
```

CHEAT SHEET

Chapter 6 — Quick Cheat Sheet

Concept	Meaning
Function	Reusable block of code
Function Declaration	<code>function greet() {}</code>
Function Call	<code>greet()</code>
Parameter	Function-এর input variable
Argument	Function call-এর actual value
<code>return</code>	Value ফেরত দেয়
Default Parameter	Default input value
Function Expression	Variable-এর মধ্যে function
Arrow Function	Short modern function syntax
Anonymous Function	নাম ছাড়া function
IIFE	Immediately executed function
Callback	Function passed to another function
Higher-Order Function	Function নেয়/return করে
Rest Parameter	Multiple arguments collect করে
Closure	Function outer scope remember করে
Recursion	Function নিজেকে call করে
Pure Function	Same input → same output
Local Variable	Function-এর ভিতরের variable
Global Variable	Outer/global scope-এর variable

MUST REMEMBER 🔥 Chapter 6 — Must Remember (Core Patterns)

Function Declaration

JAVASCRIPT

FUNCTION DECLARATION

```
function greet(name) {  
  return `Hello ${name}`;  
}
```

Call

JAVASCRIPT

CALL

```
console.log(greet("Shariar"));
```

Function Expression

JAVASCRIPT

FUNCTION EXPRESSION

```
const greet = function(name) {  
  return `Hello ${name}`;  
};
```

Arrow Function

JAVASCRIPT

ARROW FUNCTION

```
const greet = name => `Hello ${name}`;
```

Default Parameter

JAVASCRIPT

DEFAULT PARAMETER

```
function greet(name = "Guest") {  
  return `Hello ${name}`;  
}
```

Rest Parameter

JAVASCRIPT

REST PARAMETER

```
function add(...numbers) {  
  // numbers is an array  
}
```

Callback

JAVASCRIPT

CALLBACK

```
function process(callback) {  
  callback();  
}
```

Return

JAVASCRIPT

RETURN

```
function add(a, b) {  
  return a + b;  
}
```

Basic Questions

1. Function কী?
2. Function কেন ব্যবহার করা হয়?
3. Parameter এবং Argument-এর মধ্যে পার্থক্য কী?
4. `return` কী করে?
5. `console.log()` এবং `return` -এর মধ্যে পার্থক্য কী?
6. Function Declaration কী?
7. Function Expression কী?
8. Arrow Function কী?
9. Callback Function কী?
10. Higher-Order Function কী?
11. Default Parameter কী?
12. Rest Parameter কী?
13. Recursion কী?
14. Closure কী?
15. Pure Function কী?

 Coding Practice# Coding Practice

Problem 1 — Greeting Function

একটি function বানাও:

JAVASCRIPT

CODE

```
greet("Shariar");
```

Output:

```
Hello Shariar
```

Problem 2 — Add Function

JAVASCRIPT

CODE

```
add(10, 20);
```

Expected:

```
30
```

Problem 3 — Square

JAVASCRIPT

CODE

```
square(8);
```

Expected:

64

Problem 4 — Even/Odd Function

JAVASCRIPT

CODE

```
checkEvenOdd(10);
```

Expected:

Even

Problem 5 — Maximum Function

JAVASCRIPT

CODE

```
findMax(10, 25, 15);
```

Expected:

25

Problem 6 — Calculator

একটি function বানাও:

JAVASCRIPT

CODE

```
calculate(20, 5, "+");  
calculate(20, 5, "-");  
calculate(20, 5, "*");  
calculate(20, 5, "/");
```

Expected:

```
25  
15  
100  
4
```

Problem 7 — Rest Parameter

JAVASCRIPT

CODE

```
sum(10, 20, 30, 40, 50);
```

Expected:

150

Problem 8 — Callback

একটি function বানাও যেখানে callback পাঠালে callback execute হবে।

Problem 9 — Factorial

JAVASCRIPT

CODE

```
factorial(5);
```

Expected:

120

Problem 10 — Shopping Cart

এই data:

JAVASCRIPT

CODE

```
let cart = [
  { name: "Laptop", price: 70000 },
  { name: "Mouse", price: 1000 },
  { name: "Keyboard", price: 2500 }
];
```

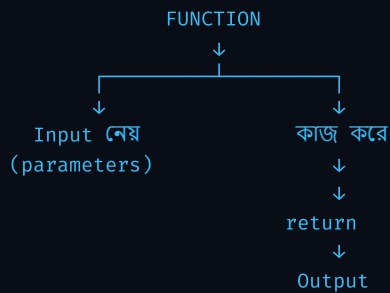
একটি function বানিয়ে total price বের করো।

Expected:

73500

SUMMARY 🚀 Chapter 6 শেষ করার পর তোমার যা পরিষ্কার হওয়া উচিত

সবচেয়ে important flow:



আর JavaScript-এর function ecosystem:

Function

- Declaration
- Expression
- Arrow Function
- Anonymous Function
- IIFE
- Callback
- Higher-Order Function
- Recursive Function
- Pure Function
- Rest Parameter
- Default Parameter
- Closure

Chapter 6-এর সবচেয়ে গুরুত্বপূর্ণ ৫টি বিষয়: `parameter vs argument` → `return` → `arrow function` → `callback` → `higher-order function`

এগুলো ভালোভাবে বুঝে গেলে পরের chapters-এর অনেক concept অনেক সহজ হয়ে যাবে।